Practical Assignment Report

Time Series Segmentation

Authors

Jesús Fernández López & Rafael Carlos Schoettel Vilchez

Course

Metaheuristics

Universidad de Córdoba

Date

March 15, 2026

Contents

1 Problem Formulation **2**

2 Algorithms: Definition and Parameter Fine-Tuning **2**
2.1 Random Search (RS) . . . 2
2.2 Hill Climbing (HC) — Steepest Ascent . . . 2
2.3 Simulated Annealing (SA) . . . 2

3 Solution Generation and Neighbourhood Structure **3**
3.1 Random Initial Solution . . . 3
3.2 Concepts of Neighbour, Neighbourhood, and Movement Operator . . . 3
3.3 Practical Example of the Operator . . . 3
3.4 Application of the Operator by Algorithm . . . 4

4 Parameter Fine-Tuning and Experimental Setup **4**
4.1 Random Search (RS): Convergence Analysis . . . 5
4.2 Hill Climbing (HC): Independent Restarts Strategy . . . 6
4.3 Simulated Annealing (SA): Grid Search Optimisation . . . 7

5 Final Evaluation and Results **8**

6 Conclusions and Critical Analysis **10**
6.1 Algorithmic Scalability and Search Space Complexity . . . 10
6.2 Objective Function Vulnerability: The "MSE Hack" . . . 10

7 Annex: Algorithmic Behaviour Visualisations **11**
7.1 General Behaviour on TS1 ($k = 9$) . . . 12
7.2 General Behaviour on TS2 ($k = 10$) . . . 13
7.3 General Behaviour on TS3 ($k = 20$) . . . 14
7.4 General Behaviour on TS4 ($k = 50$) . . . 15

1 Problem Formulation

The goal is to segment a time series into k segments. For each segment, a linear regression must be fitted using all the points within that segment. The aim is to determine the k segments (i.e., the bounds of each segment) that minimise the average Mean Squared Error across all segments.

- **Solution Representation:** A solution is defined as an array of $k-1$ sorted integers representing the indices (time steps) at which the series is split.
- **Objective Function (Fitness):** Minimise the Average Mean Squared Error across the k segments. For each segment, a linear regression ($y = mx + b$) is fitted and its residual error is evaluated:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \hat{y}_i\big)^2. \tag{1}$$

  The total cost of a solution is the arithmetic mean of the individual segment MSEs.
- **Constraints:**
  1. Split points must lie strictly within the series bounds ($0 < \text{split} < N$).
  2. No duplicate split points are allowed within the same solution.
  3. Split points must be sorted in ascending order.

2 Algorithms: Definition and Parameter Fine-Tuning

Before detailing how the search space is navigated, it is necessary to define the three algorithms applied in this study and the methodology used to tune their parameters for optimal performance across different time series (TSx).

2.1 Random Search (RS)

Definition: An uninformed algorithm that uniformly samples the search space. Instead of exploiting a known neighbourhood, it generates a completely new, independent random solution at each iteration.

2.2 Hill Climbing (HC) — Steepest Ascent

Definition: A purely exploitative local search strategy. At each step, it exhaustively evaluates all possible neighbouring solutions and strictly transitions to the neighbour that provides the greatest reduction in MSE.

2.3 Simulated Annealing (SA)

Definition: A probabilistic metaheuristic inspired by the annealing process in metallurgy. It avoids premature convergence by occasionally accepting transitions to worse solutions. This acceptance is governed by the Metropolis Criterion ($P = e^{-\Delta E/T}$), which depends on a *temperature* parameter that decreases over time according to a cooling schedule.

3 Solution Generation and Neighbourhood Structure

To navigate the search space (composed of all valid combinations of $k-1$ split points), it is essential to define how solutions are built and how the algorithm moves between them.

3.1 Random Initial Solution

To begin the search, we generate an initial solution S_0. We randomly select $k-1$ unique points within the available range $[1, N-1]$ and sort them in ascending order. This ensures that all points have the same initial probability of being chosen, avoiding any bias in the initial segment sizes.

3.2 Concepts of Neighbour, Neighbourhood, and Movement Operator

In combinatorial optimisation, a **neighbour** (S') is a new solution obtained by applying a small modification to the current solution (S). The **neighbourhood** is the complete set of all valid neighbours reachable in a single step from the current state.

To generate these neighbours, we have designed an **Atomic Movement Operator (Δ-move)**. The operation consists of choosing a single split point s_i from the current solution and shifting it by an integer distance Δ.

- The magnitude of Δ is bounded by a maximum-variability parameter (`max_var`), empirically set to 1% of the total number of points N. Thus $\Delta \in [-\texttt{max_var}, +\texttt{max_var}]$, with $\Delta \neq 0$.
- **Validity Constraints:** For the neighbour to be valid, the new split point ($s_i + \Delta$) must satisfy $0 < s_i + \Delta < N$ and must not coincide with any existing split point in the solution.

3.3 Practical Example of the Operator

Consider a time series of $N = 300$ points divided into $k = 3$ segments (requiring $k - 1 = 2$ split points). Suppose the current solution is $S = [50,\ 150]$. With $\texttt{max_var} = \lfloor 0.01 \times 300 \rfloor = 3$, the operator evaluates **every integer** $\Delta \in [-3, 3] \setminus \{0\}$, systematically generating the following complete **neighbourhood**:

1. **Movements applied to the first split point (50), $\Delta \in \{-3, -2, -1, 1, 2, 3\}$:**
   - $\Delta = -3 \ \rightarrow\ S'_1 = [47,\ 150]$
   - $\Delta = -2 \ \rightarrow\ S'_2 = [48,\ 150]$
   - $\Delta = -1 \ \rightarrow\ S'_3 = [49,\ 150]$
   - $\Delta = +1 \ \rightarrow\ S'_4 = [51,\ 150]$
   - $\Delta = +2 \ \rightarrow\ S'_5 = [52,\ 150]$
   - $\Delta = +3 \ \rightarrow\ S'_6 = [53,\ 150]$

2. **Movements applied to the second split point (150), $\Delta \in \{-3, -2, -1, 1, 2, 3\}$:**
   - $\Delta = -3 \;\rightarrow\; S'_7 = [50,\ 147]$
   - $\Delta = -2 \;\rightarrow\; S'_8 = [50,\ 148]$
   - $\Delta = -1 \;\rightarrow\; S'_9 = [50,\ 149]$
   - $\Delta = +1 \;\rightarrow\; S'_{10} = [50,\ 151]$
   - $\Delta = +2 \;\rightarrow\; S'_{11} = [50,\ 152]$
   - $\Delta = +3 \;\rightarrow\; S'_{12} = [50,\ 153]$

In a single step, the neighbourhood is deterministically composed of these 12 distinct solutions, demonstrating that the operator explores the entire integer range rather than only its boundaries.

3.4 Application of the Operator by Algorithm

The way each algorithm interacts with this operator defines its exploration and exploitation strategy:

- **Random Search (RS):** *Does not use the movement operator or the concept of a neighbourhood.* At each iteration it discards the current solution and generates a completely new solution S' from scratch. It performs blind global sampling without exploiting any local information.

- **Hill Climbing — Steepest Ascent:** Uses the operator to generate and evaluate the **entire neighbourhood** at each iteration. In the previous example, it would evaluate all 12 generated neighbours and strictly select the one that minimises MSE. If no neighbour improves upon S, the algorithm concludes that a local optimum has been reached and terminates.

- **Simulated Annealing (SA):** Uses the operator to generate a **single random neighbour** per iteration. It randomly selects one split point and applies a random Δ from the allowed range (e.g., selects point 150 and applies +2, producing only S'_{11}). The decision to accept this new solution is governed by the Metropolis Criterion, allowing the algorithm to temporarily accept a worse MSE in order to escape local minima in a temperature-controlled manner.

4 Parameter Fine-Tuning and Experimental Setup

To ensure a fair and rigorous comparison between the metaheuristics, a preliminary study was conducted to fine-tune each algorithm's parameters. The primary goal was to find the optimal balance between solution quality (measured by MSE) and computational cost for each specific time series.

4.1 Random Search (RS): Convergence Analysis

As a purely stochastic algorithm with no heuristic guidance, Random Search relies entirely on the volume of iterations to find high-quality solutions. To avoid an arbitrary stopping criterion, a multi-run convergence study was performed.

The algorithm was executed multiple times; the average best MSE at each iteration was tracked to mitigate the impact of random variance. The theoretical optimal stopping point was identified as the iteration where the average convergence curve exhibited clear asymptotic behaviour (reaching within a 2% tolerance of the absolute minimum found).

The empirical optimal average stops were 6,869, 9,021, 4,369, and 3,445 iterations for TS1, TS2, TS3, and TS4, respectively.

However, relying strictly on the exact average limit leaves the algorithm vulnerable to premature termination in exceptionally unlucky runs. Therefore, a deliberate safety margin was added. The final stopping criteria adopted were:

- **TS1** ($k = 9$): 7,000 iterations.
- **TS2** ($k = 10$): 9,250 iterations.
- **TS3** ($k = 20$): 4,500 iterations.
- **TS4** ($k = 50$): 3,600 iterations.

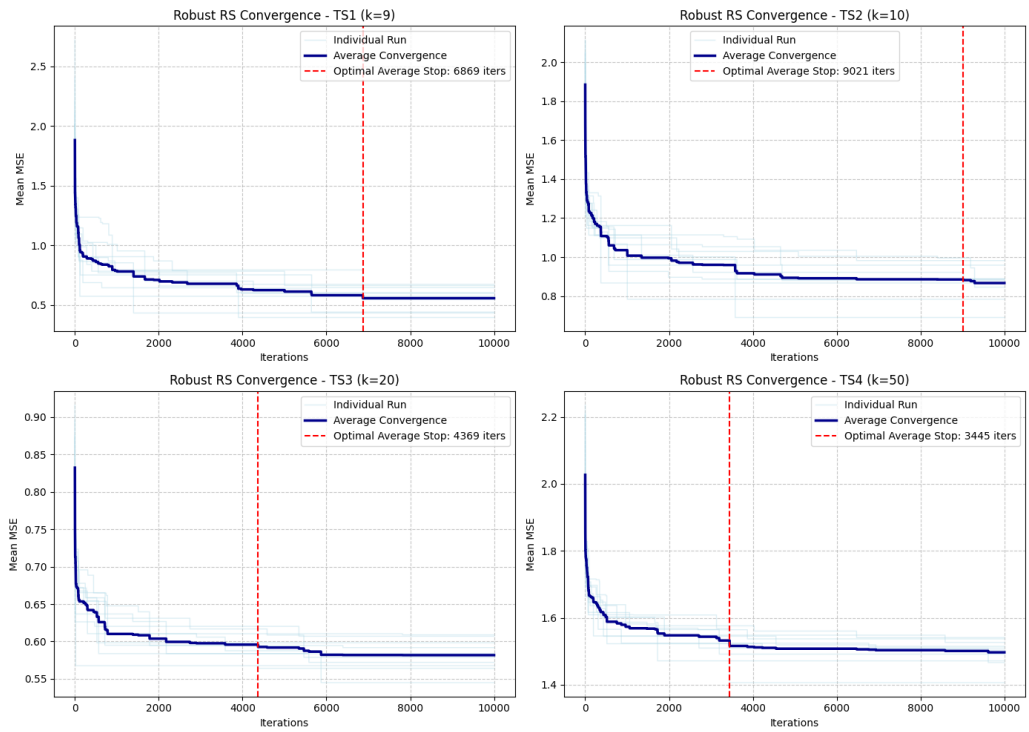

Figure 1: Random Search multi-run average convergence analysis.

4.2 Hill Climbing (HC): Independent Restarts Strategy

Hill Climbing is a strictly exploitative local search algorithm. Once initialised, its deterministic nature ensures it will rapidly descend into the nearest local optimum, where it becomes permanently trapped. Consequently, internal parameter tuning (e.g., cooling rates) is not applicable to its core mechanism.

To evaluate its true capability as a global optimiser, a multi-restart strategy was implemented. The central design question was determining how many random initialisations are required to sufficiently sample different valleys of the search space.

A preliminary study tracked the cumulative best MSE across sequential independent restarts. To avoid subjective visual estimation, a strict mathematical tolerance threshold was applied: the optimal stopping point was defined as the exact number of restarts for which the marginal improvement falls below 1% of the absolute minimum found across all trials.

This mathematically rigorous approach guarantees that the algorithm has extensively explored the search space and converged to a near-global optimum. The final restart configurations were:

- **TS1** ($k = 9$): 18 restarts.
- **TS2** ($k = 10$): 13 restarts.
- **TS3** ($k = 20$): 9 restarts.
- **TS4** ($k = 50$): 18 restarts.

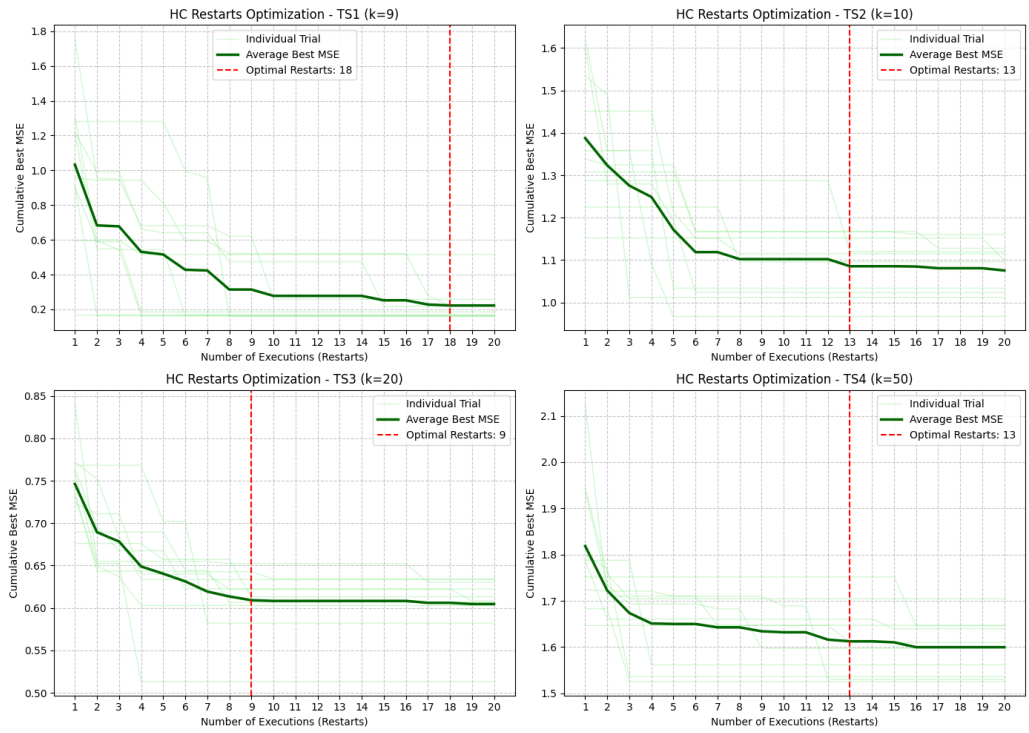

Figure 2: Hill Climbing cumulative best MSE across independent restarts.

4.3 Simulated Annealing (SA): Grid Search Optimisation

Simulated Annealing demands precise tuning of its cooling schedule to properly manage the transition from broad exploration (high temperature) to strict exploitation (low temperature). A robust Grid Search methodology was applied to determine the optimal configuration.

The study evaluated Geometric and Linear cooling schedules across various initial temperatures ($T_0 \in \{50, 100\}$), cooling rates ($\alpha \in \{0.95, 0.98, 0.99\}$ for geometric; step sizes $\{0.5, 1.0\}$ for linear), and epochs per temperature step ($L \in \{50, 100\}$).

Decision note: The Logarithmic cooling schedule, despite its theoretical guarantees of global convergence, was empirically excluded early in the process. Initial tests revealed that its extremely slow temperature reduction massively exceeded any reasonable computational time limits, rendering it impractical for these specific time series problems.

Table 1 summarises the optimal parameter combinations discovered. In all scenarios, the Geometric cooling schedule vastly outperformed the Linear approach, providing a smoother transition to thermal equilibrium.

Table 1: Optimal Simulated Annealing parameters obtained via Grid Search.

Dataset	**Cooling**	T_0	α	L **(Epochs)**	**Best MSE**
TS1 ($k = 9$)	Geometric	100	0.98	100	**0.1591**
TS2 ($k = 10$)	Geometric	50	0.98	100	**0.8529**
TS3 ($k = 20$)	Geometric	50	0.99	100	**0.2293**
TS4 ($k = 50$)	Geometric	100	0.99	100	**0.4670**

The Grid Search results validate a core principle of metaheuristic design: parameter selection is highly dependent on problem scale. While a high number of internal evaluations ($L = 100$) proved beneficial across all datasets, the cooling rate (α) varied with problem complexity. For larger search spaces (TS3 and TS4), the algorithm systematically favoured a slower cooling rate ($\alpha = 0.99$) to prolong thermal equilibrium. For simpler landscapes (TS1 and TS2), a faster rate ($\alpha = 0.98$) was sufficient without unnecessary computational overhead.

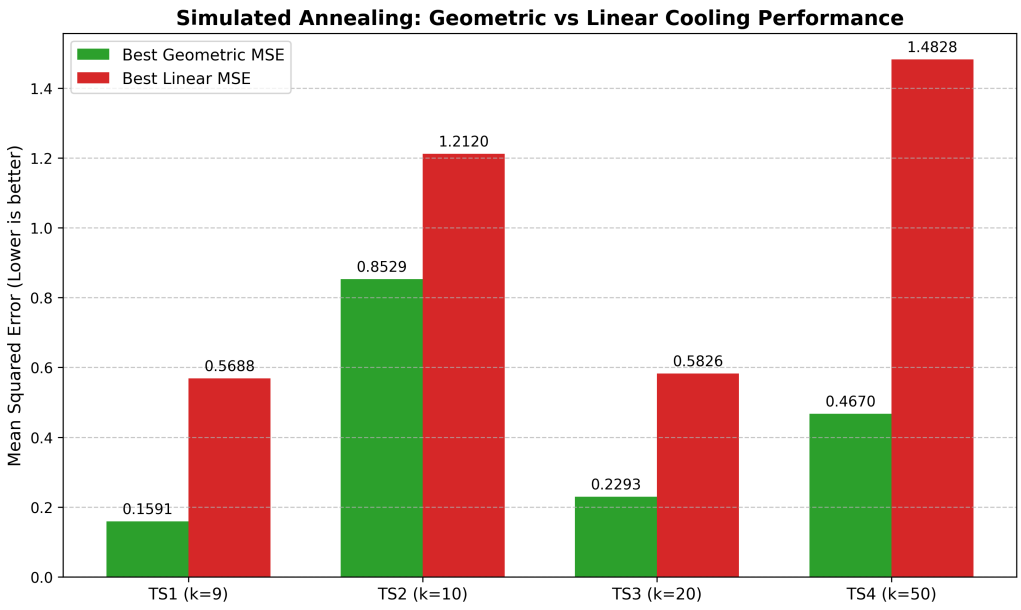

Figure 3: Performance comparison: best Geometric vs. best Linear cooling MSE.

5 Final Evaluation and Results

Following the rigorous fine-tuning phase, a final execution was performed using the optimal parameters established for each algorithm. To determine the absolute best segmentation for each time series, the algorithms were evaluated across multiple independent runs, retaining only the single best solution found.

Table 2 summarises the quantitative performance of the winning algorithm for each dataset, illustrating the relationship between search space dimensionality (defined by k) and algorithmic suitability.

Table 2: Summary of the best solutions found across all datasets.

Dataset	Segments (k)	Winning Algorithm	Best MSE	Execution Time (s)
TS1	9	Hill Climbing	0.1591	0.04
TS2	10	Simulated Annealing	0.8164	2.15
TS3	20	Simulated Annealing	0.2222	5.61
TS4	50	Simulated Annealing	0.4435	13.90

As observed in the summary, Hill Climbing provides a remarkably fast and competitive solution for lower-complexity landscapes (TS1). However, as the number of segments increases (TS3 and TS4), Simulated Annealing consistently achieves the best results, effectively navigating the combinatorial explosion at a naturally higher computational cost.

The following figures visually present these best segmentations. Notably, Figure 5 exposes the mathematical exploit discovered in TS2, where Simulated Annealing clustered split points at the edges to artificially minimise the unconstrained objective function.

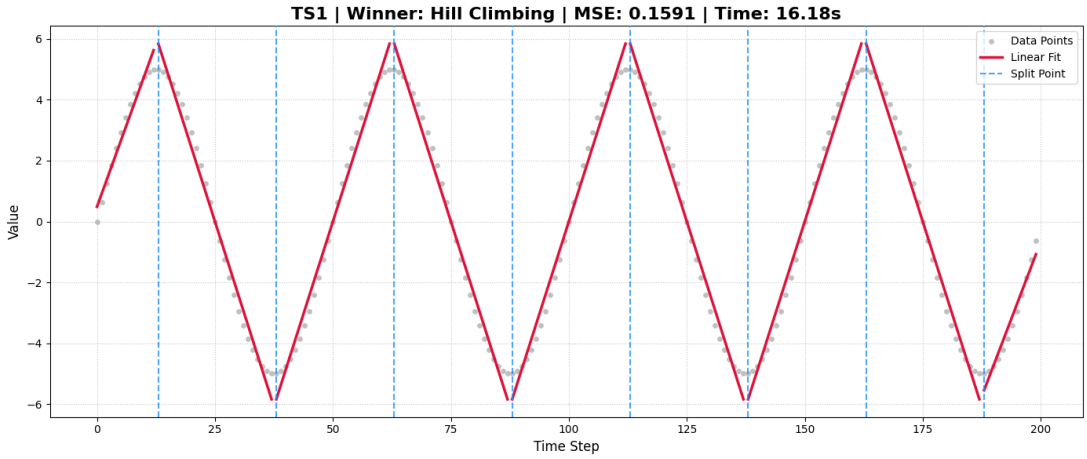

Figure 4: Best solution for TS1. Winner: Hill Climbing.

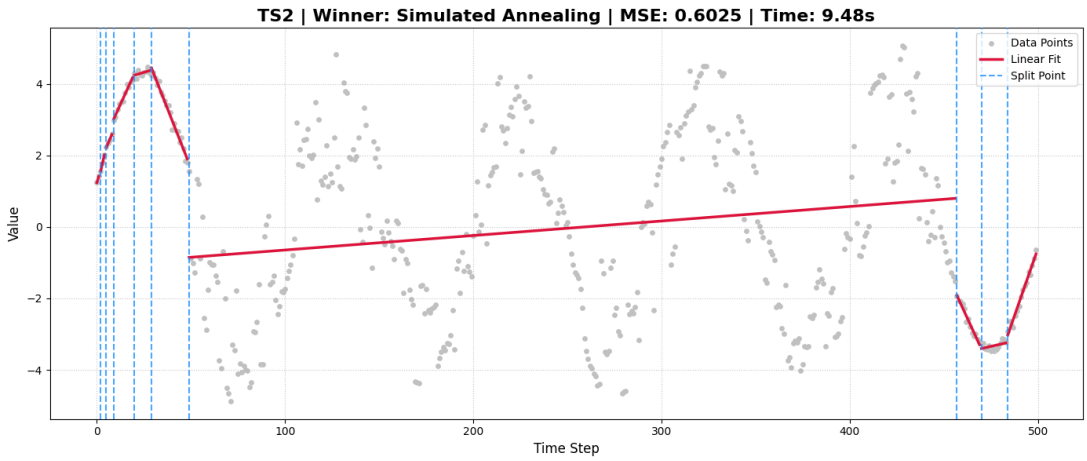

Figure 5: Best solution for TS2. Winner: Simulated Annealing (note the split-point clustering at the edges).

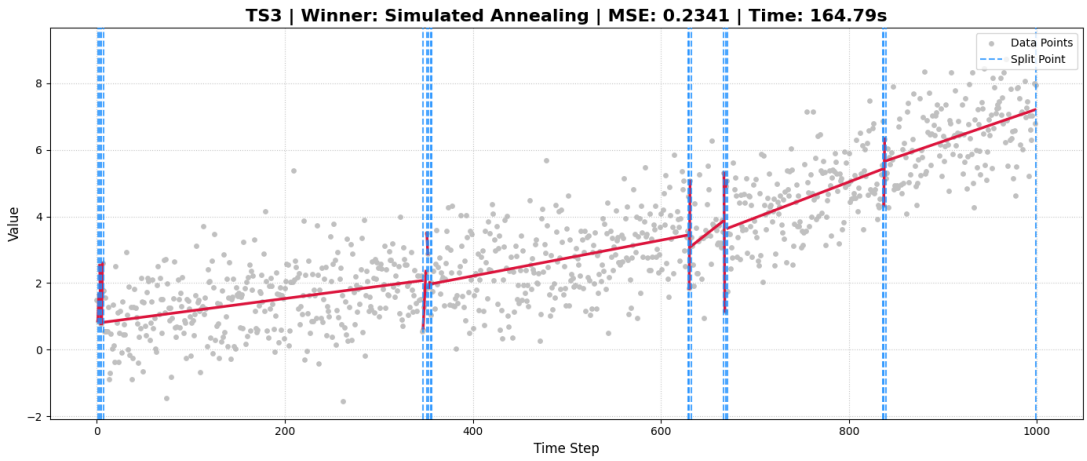

Figure 6: Best solution for TS3. Winner: Simulated Annealing.

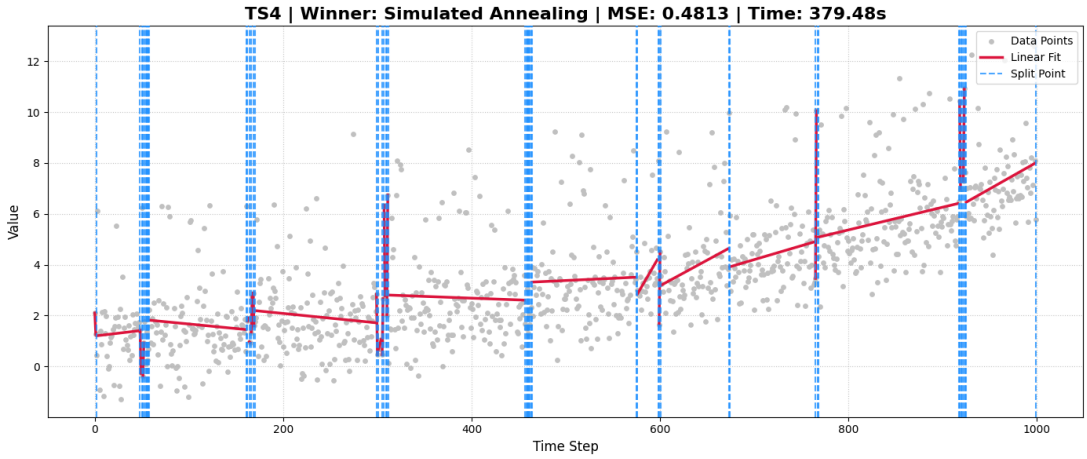

Figure 7: Best solution for TS4. Winner: Simulated Annealing.

6 Conclusions and Critical Analysis

The experimental results yield two major conclusions regarding algorithmic scalability and objective function design in time series segmentation.

6.1 Algorithmic Scalability and Search Space Complexity

In relatively simple optimisation problems with low dimensionality, such as TS1 ($k = 9$), any well-implemented metaheuristic can perform adequately. In our final trial, Hill Climbing achieved the lowest MSE (0.1591). However, in such constrained spaces the performance difference is marginal: the "winner" could have equally been Simulated Annealing or even Random Search, since success in low-complexity landscapes is heavily influenced by stochastic initialisation rather than algorithmic superiority.

Conversely, as problem complexity scales exponentially (TS3 with $k = 20$ and TS4 with $k = 50$), the behaviour changes drastically. Algorithms that lack a robust equilibrium between *intensification* (exploitation) and *diversification* (exploration) fail entirely. Hill Climbing repeatedly becomes trapped in sub-optimal local minima due to its greedy nature; Random Search lacks the heuristic guidance to refine boundaries. Simulated Annealing clearly dominates these high-dimensional scenarios. Its thermodynamic cooling schedule and stochastic acceptance criterion perfectly balance the exploration of new regions with the fine-tuning of local optima, consistently securing the best MSEs.

6.2 Objective Function Vulnerability: The "MSE Hack"

The results for TS2 (Figure 5) expose a critical vulnerability in relying solely on an unweighted arithmetic mean of MSEs as the fitness function. While Simulated Annealing mathematically "won" by achieving an MSE of 0.8164, visual inspection reveals that the resulting segmentation completely fails to represent the actual curve of the data.

Because a linear regression fitted to one or two points yields an error of exactly zero, the highly explorative SA algorithm effectively "hacked" the objective function. It achieved this by clustering almost all split points at the extreme edges of the dataset, creating numerous microscopic segments with zero error. Since the fitness function calculates an

unweighted arithmetic mean, these zero-error segments dragged the overall average down, successfully masking the massive residual error of the single oversized segment spanning the middle of the dataset.

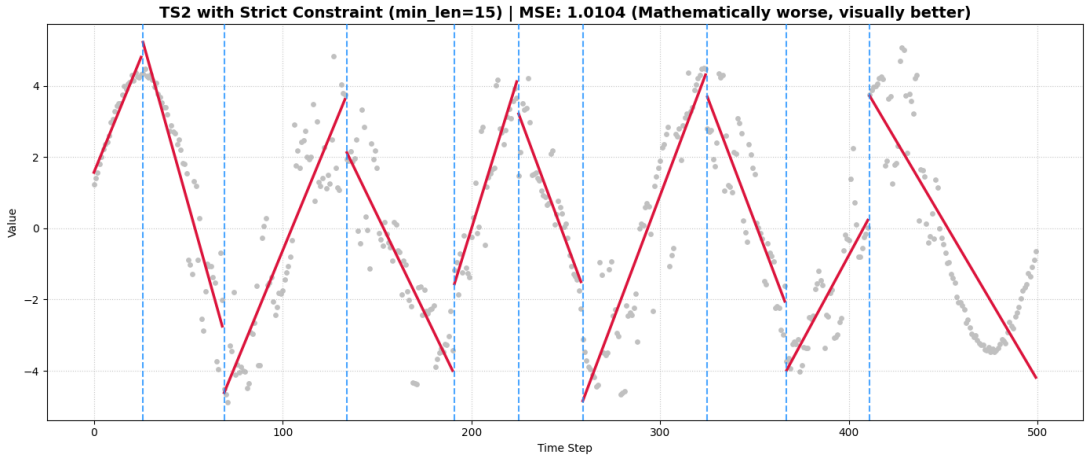

Figure 8: Visual proof of the objective function vulnerability. By enforcing a minimum segment length constraint (`min_len` = 15), the algorithm is forced to follow the actual data curve. While this produces a visually superior and practically useful segmentation, it paradoxically yields a mathematically "worse" MSE (1.1098 vs. 0.8164).

To generate solutions that genuinely track the underlying data and prevent this exploit, the fitness evaluation must be structurally constrained. Future implementations could address this by:

1. Imposing a strict **minimum segment length** constraint during neighbour generation, forcing the algorithm to distribute cuts more evenly (as demonstrated in Figure 8).

2. Replacing the arithmetic mean with a **geometric mean** or a **weighted average** based on the number of points in each segment, preventing tiny segments from disproportionately lowering the overall score.

3. Incorporating **standard deviation penalties** into the fitness function, penalising solutions that exhibit extreme variance in segment sizes.

7 Annex: Algorithmic Behaviour Visualisations

Note: The following visualisations were generated during early developmental phases, prior to optimal parameter tuning. While their quantitative metrics do not represent the final maximised performance of the algorithms, they are preserved here as qualitative examples of the inherent behavioural differences between purely stochastic exploration (Random Search), strict local exploitation (Hill Climbing), and balanced thermodynamic search (Simulated Annealing).

7.1 General Behaviour on TS1 ($k = 9$)

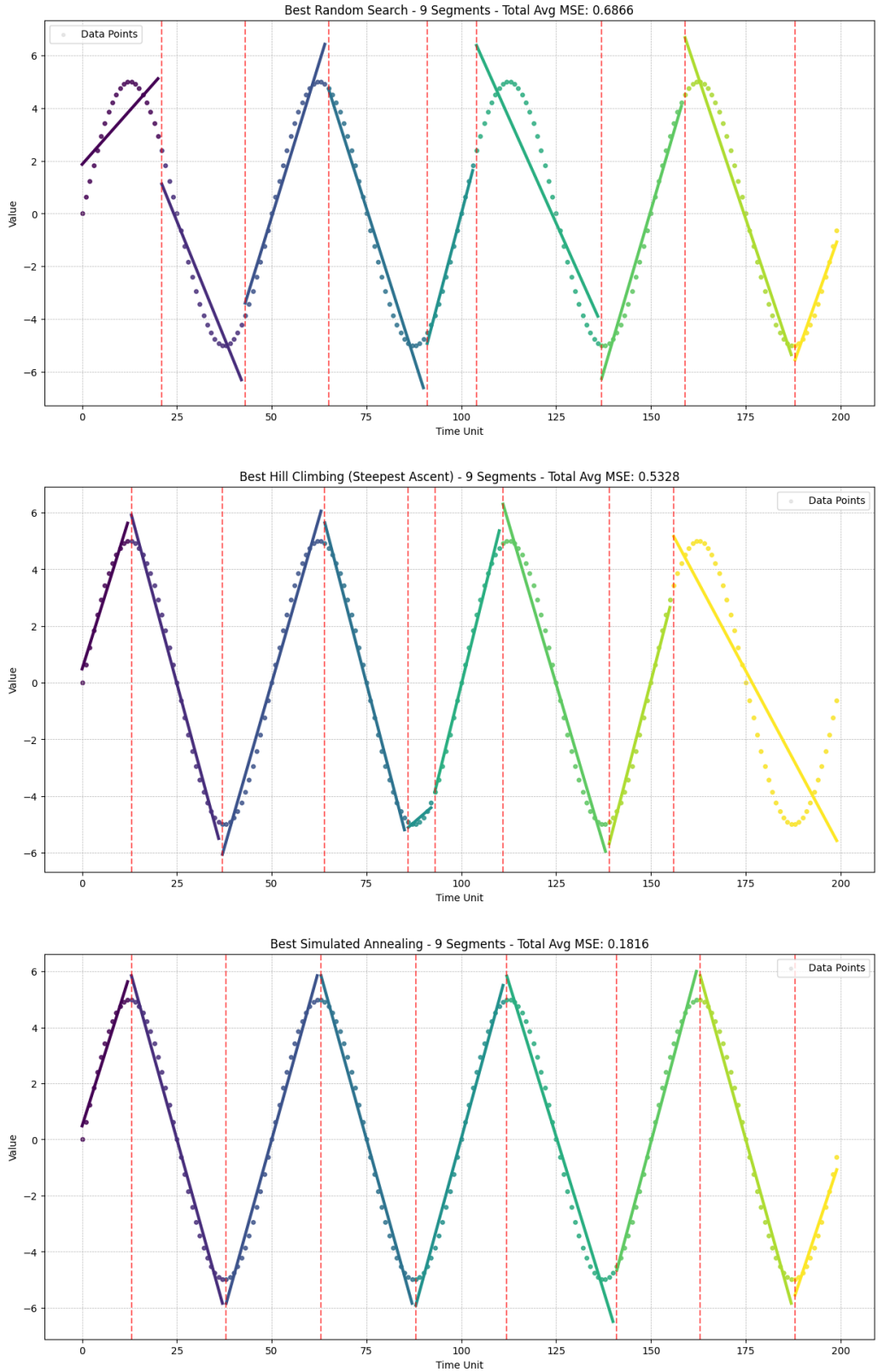

Figure 9: Behavioural comparison on TS1. Top: Random Search. Middle: Hill Climbing. Bottom: Simulated Annealing.

7.2 General Behaviour on TS2 ($k = 10$)

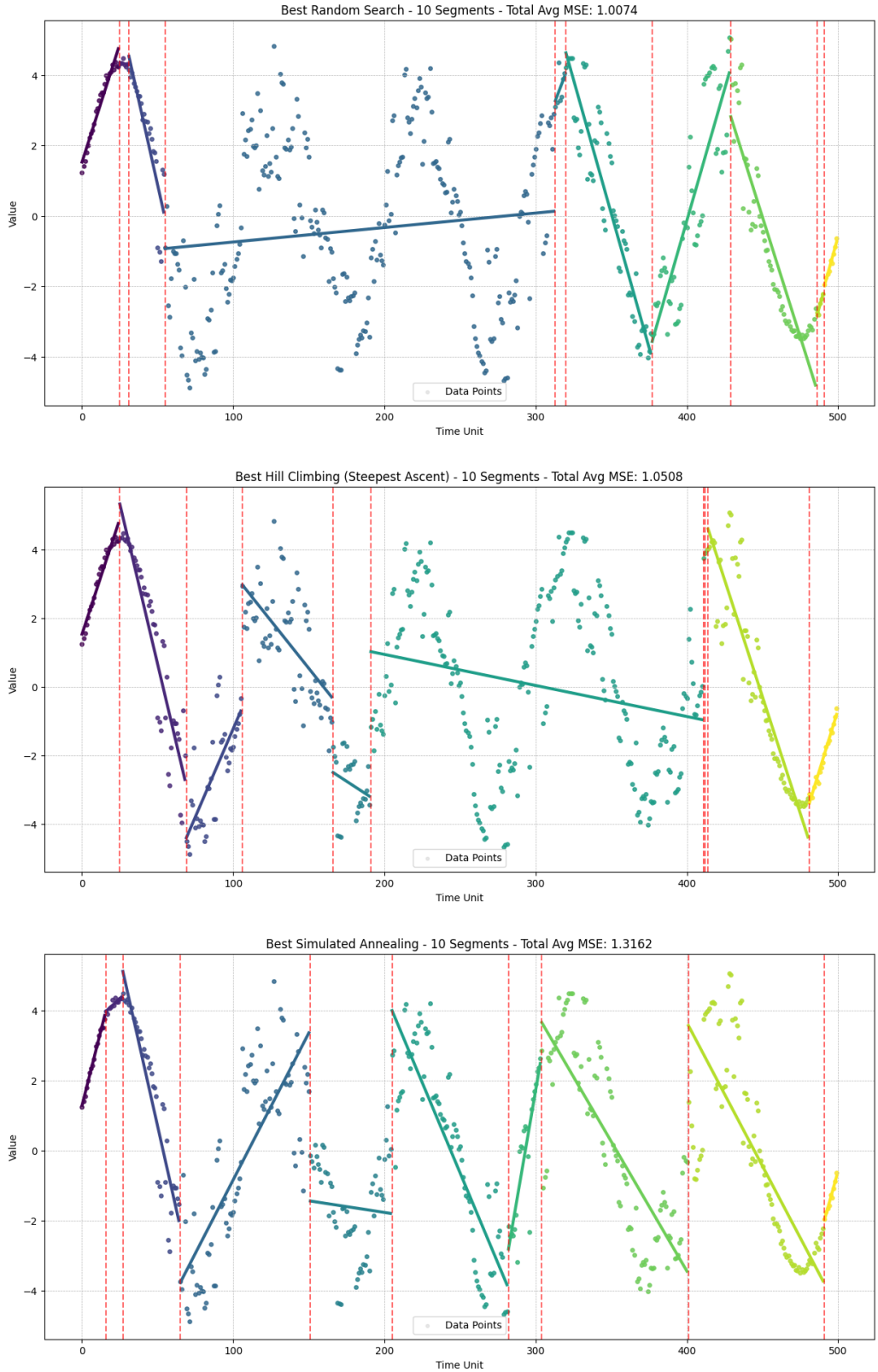

Figure 10: Behavioural comparison on TS2. Top: Random Search. Middle: Hill Climbing. Bottom: Simulated Annealing.

7.3 General Behaviour on TS3 ($k = 20$)

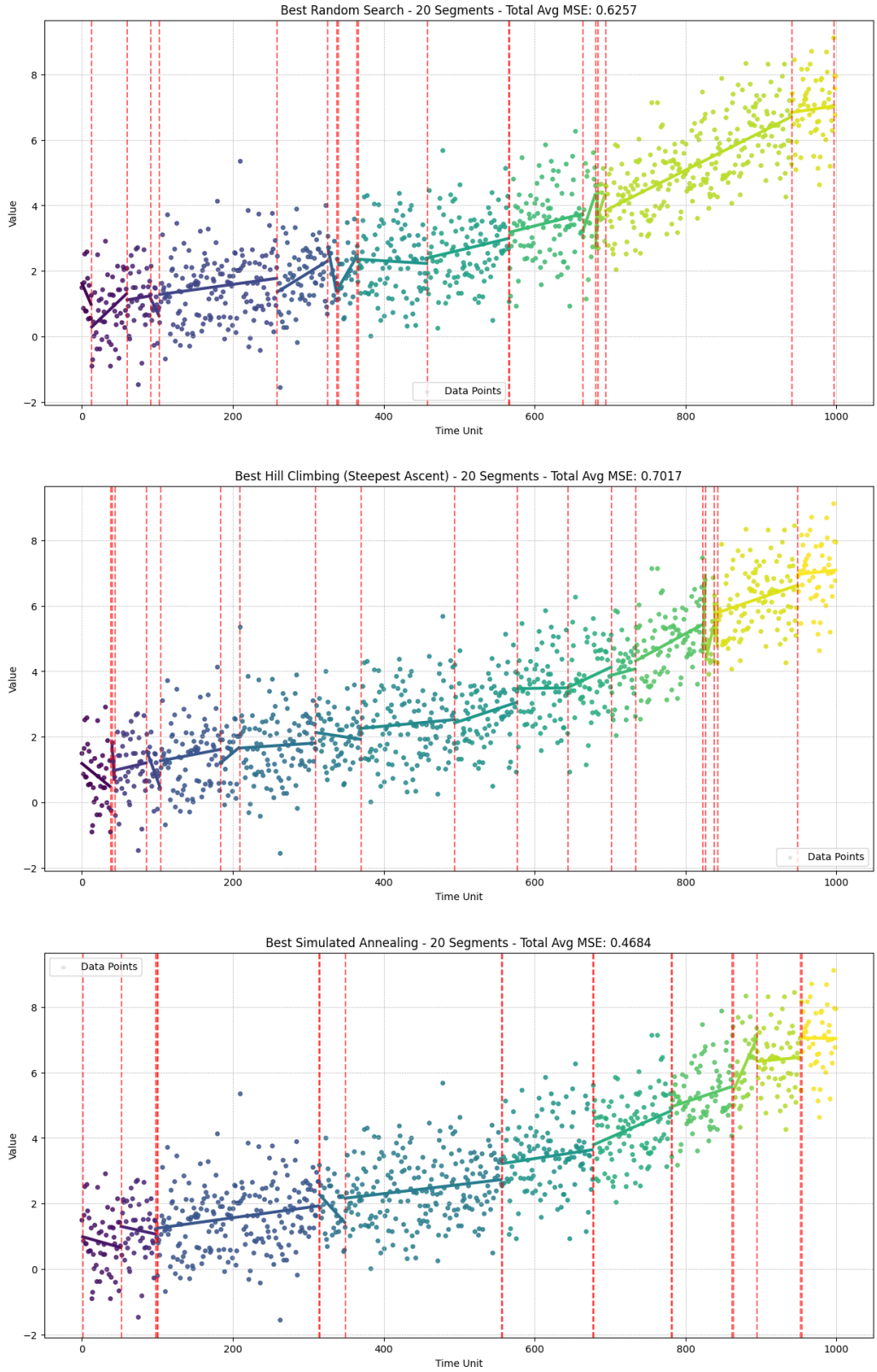

Figure 11: Behavioural comparison on TS3. Top: Random Search. Middle: Hill Climbing. Bottom: Simulated Annealing.

7.4 General Behaviour on TS4 ($k = 50$)

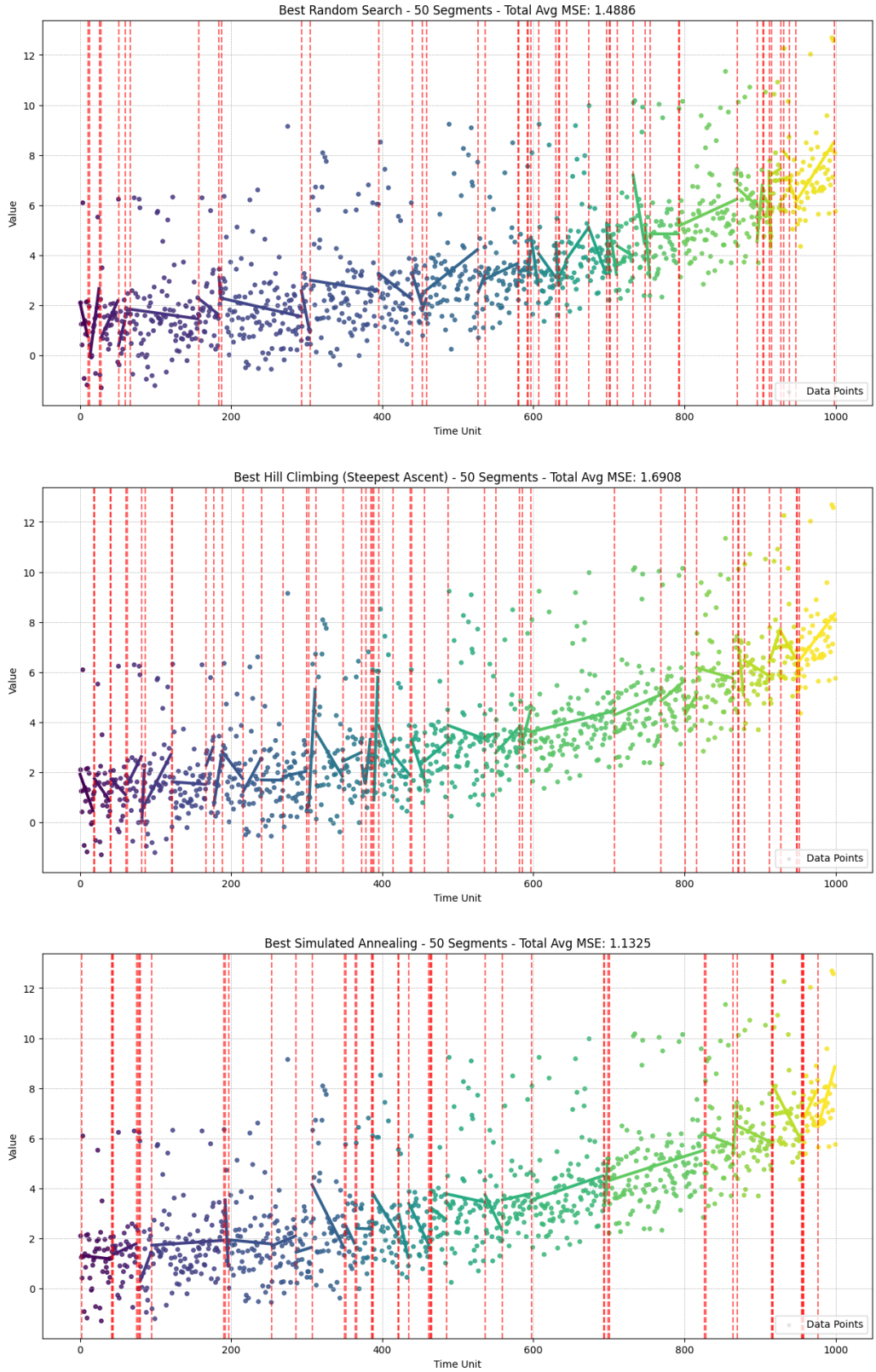

Figure 12: Behavioural comparison on TS4. Top: Random Search. Middle: Hill Climbing. Bottom: Simulated Annealing.